

《星溯旅人》第二阶段：核心引擎开发 - 详细分步操作指南

部分内容由豆包生成

 **本阶段目标：**完善核心引擎各系统的具体实现，建立事件驱动架构，实现场景生命周期管理，完善资源、输入、音频、配置、存档等系统。

预计耗时：2-3周

前置条件：第一阶段项目初始化已完成，项目能正常编译运行

一、第二阶段概述

1.1 阶段目标

第二阶段的核心任务是将第一阶段搭建的基础框架，完善为可实际使用的游戏引擎。重点是建立**事件驱动架构**，实现各系统间的解耦通信，完善各子系统的具体功能。

1.2 主要工作内容

序号	系统	主要工作
1	游戏主循环	实现deltaTime、固定时间步长、FPS计数、时间缩放
2	事件系统	事件总线、事件订阅/发布、常用事件类型定义
3	场景系统	场景基类、生命周期管理、场景切换过渡
4	资源管理	引用计数、异步加载、资源队列、缓存策略
5	输入系统	输入上下文、输入事件、配置文件支持
6	音频系统	音频池、淡入淡出、音频组、3D音效基础
7	配置与存档	配置联动、存档结构完善、版本管理
8	插件系统	插件接口定义、插件加载与管理
9	集成测试	各系统联调、测试场景、性能基准

1.3 阶段交付物

- 完善的核心引擎（8个系统全部可用）
- 事件驱动架构（各系统解耦）
- 可运行的测试场景
- 基础调试工具
- 性能基准报告

二、步骤1：完善游戏主循环和时间管理



目标：实现精确的游戏时间管理，包括deltaTime计算、固定时间步长、FPS计数、时间缩放功能

修改文件：gameEngine.h / gameEngine.cpp

新增文件：timeManager.h / timeManager.cpp（可选，也可直接在GameEngine中实现）

1.1 为什么需要完善时间管理

第一阶段的GameEngine使用QTimer实现固定间隔的主循环，但缺少游戏开发中关键的时间管理功能：

- **deltaTime：**两帧之间的时间差，用于帧率无关的运动计算
- **固定时间步长：**物理和逻辑更新使用固定时间间隔，确保一致性
- **FPS计数：**实时帧率统计，用于性能监控
- **时间缩放：**慢动作、暂停等效果的基础

1.2 修改GameEngine头文件

操作步骤

1. 在Qt Creator中打开 `src/core/gameEngine.h`
2. 在private成员变量区域，添加时间管理相关的成员
3. 添加时间管理相关的公有方法
4. 保存文件（Ctrl+S）

完整修改后的gameEngine.h（时间管理部分）

gameEngine.h - 新增时间管理成员

```
1  // 在类的public区域添加:
2
3  // 时间管理
4  float getDeltaTime() const;
5  float getUnscaledDeltaTime() const;
6  float getTimeScale() const;
7  void setTimeScale(float scale);
8  float getFPS() const;
9  float getAverageFPS() const;
10 float getGameTime() const;
11 bool isPaused() const;
12 void setPaused(bool paused);
13
14 // 在类的private区域添加:
15
16 // 时间管理
17 float m_deltaTime;           // 缩放后的帧时间（秒）
18 float m_unscaledDeltaTime;   // 原始帧时间（秒）
19 float m_timeScale;           // 时间缩放系数
20 float m_gameTime;            // 游戏运行时间（缩放后）
21 float m_realTime;            // 真实运行时间（未缩放）
22 bool m_isPaused;             // 是否暂停
23
24 // FPS计数
25 float m_fps;                 // 当前FPS
26 float m_averageFPS;          // 平均FPS
27 float m_fpsAccumulator;      // FPS累加器
28 int m_fpsFrameCount;         // FPS帧计数器
29 static constexpr float FPS_UPDATE_INTERVAL = 0.5f; // FPS更新间隔（秒）
30
31 // 固定时间步长
32 float m_fixedTimestep;        // 固定时间步长（秒）
33 float m_fixedAccumulator;     // 固定步长累加器
34 static constexpr float DEFAULT_FIXED_TIMESTEP = 1.0f / 60.0f; // 默认60次/秒
35
36 // 时间记录
37 QElapsedTimer* m_frameTimer;  // 帧计时器
38 qint64 m_lastFrameTime;      // 上一帧时间（纳秒）
```

注意：需要在头文件顶部添加 `#include <QElapsedTimer>`

1.3 修改GameEngine实现文件

操作步骤

1. 在Qt Creator中打开 `src/core/gameEngine.cpp`
2. 在构造函数中初始化时间管理成员
3. 修改initialize()方法，初始化帧计时器
4. 重写update()方法，实现完整的时间管理逻辑
5. 添加所有时间管理方法的实现
6. 保存文件（Ctrl+S）

构造函数初始化

gameEngine.cpp - 构造函数新增

```
1  // 在构造函数初始化列表中添加：
2  , m_deltaTime(0.0f)
3  , m_unscaledDeltaTime(0.0f)
4  , m_timeScale(1.0f)
5  , m_gameTime(0.0f)
6  , m_realTime(0.0f)
7  , m_isPaused(false)
8  , m_fps(0.0f)
9  , m_averageFPS(0.0f)
10 , m_fpsAccumulator(0.0f)
11 , m_fpsFrameCount(0)
12 , m_fixedTimestep(DEFAULT_FIXED_TIMESTEP)
13 , m_fixedAccumulator(0.0f)
14 , m_frameTimer(new QElapsedTimer())
15 , m_lastFrameTime(0)
```

initialize()方法修改

gameEngine.cpp - initialize新增

```
1  // 在initialize()方法开头添加：
2  m_frameTimer->start();
3  m_lastFrameTime = m_frameTimer->nsecsElapsed();
```

重写update()方法

gameEngine.cpp - 完整的update方法

```
1  void GameEngine::update()
2  {
```

```
3 // 1. 计算帧时间
4 qint64 currentTime = m_frameTimer->nsecsElapsed();
5 qint64 deltaNsec = currentTime - m_lastFrameTime;
6 m_lastFrameTime = currentTime;
7
8 // 转换为秒 (纳秒 -> 秒)
9 m_unscaledDeltaTime = static_cast<float>(deltaNsec) / 1000000000.0f;
10
11 // 防止卡顿导致的超大deltaTime (限制最大0.1秒)
12 if (m_unscaledDeltaTime > 0.1f) {
13     m_unscaledDeltaTime = 0.1f;
14 }
15
16 // 应用时间缩放
17 if (m_isPaused) {
18     m_deltaTime = 0.0f;
19 } else {
20     m_deltaTime = m_unscaledDeltaTime * m_timeScale;
21 }
22
23 // 更新时间计数器
24 m_realTime += m_unscaledDeltaTime;
25 m_gameTime += m_deltaTime;
26
27 // 2. FPS计数
28 m_fpsFrameCount++;
29 m_fpsAccumulator += m_unscaledDeltaTime;
30 if (m_fpsAccumulator >= FPS_UPDATE_INTERVAL) {
31     m_fps = static_cast<float>(m_fpsFrameCount) / m_fpsAccumulator;
32     // 平滑平均FPS
33     if (m_averageFPS <= 0.0f) {
34         m_averageFPS = m_fps;
35     } else {
36         m_averageFPS = m_averageFPS * 0.9f + m_fps * 0.1f;
37     }
38     m_fpsFrameCount = 0;
39     m_fpsAccumulator = 0.0f;
40 }
41
42 // 3. 固定时间步长更新 (物理、逻辑等)
43 if (!m_isPaused) {
44     m_fixedAccumulator += m_deltaTime;
45     while (m_fixedAccumulator >= m_fixedTimestep) {
46         fixedUpdate(m_fixedTimestep);
47         m_fixedAccumulator -= m_fixedTimestep;
48     }
49 }
```

```
50
51     // 4. 可变时间步长更新（渲染、输入等）
52     variableUpdate(m_deltaTime);
53
54     // 5. 渲染
55     render();
56 }
57
58 void GameEngine::fixedUpdate(float deltaTime)
59 {
60     Q_UNUSED(deltaTime);
61     // TODO: 固定时间步长的逻辑更新
62     // - 物理模拟
63     // - AI更新
64     // - 游戏逻辑
65 }
66
67 void GameEngine::variableUpdate(float deltaTime)
68 {
69     Q_UNUSED(deltaTime);
70     // TODO: 可变时间步长的更新
71     // - 输入处理
72     // - 相机更新
73     // - 动画更新
74     // - UI更新
75
76     // 更新输入管理器
77     if (m_inputManager) {
78         m_inputManager->update();
79     }
80
81     // 更新场景
82     if (m_sceneManager) {
83         m_sceneManager->updateCurrentScene(deltaTime);
84     }
85 }
86
87 void GameEngine::render()
88 {
89     // TODO: 渲染逻辑
90     // Qt的QGraphicsView会自动渲染，这里主要是触发更新
91     if (m_sceneManager && m_sceneManager->getView()) {
92         m_sceneManager->getView()->viewport()->update();
93     }
94 }
```

注意：需要在gameEngine.h中添加 `fixedUpdate()`、`variableUpdate()`、`render()` 三个私有方法的声明。

时间管理方法实现

gameEngine.cpp - 时间管理方法

```
1  // 时间管理方法实现
2  float GameEngine::getDeltaTime() const { return m_deltaTime; }
3  float GameEngine::getUnscaledDeltaTime() const { return m_unscaledDeltaTime; }
4  float GameEngine::getTimeScale() const { return m_timeScale; }
5
6  void GameEngine::setTimeScale(float scale)
7  {
8      m_timeScale = qMax(0.0f, scale);
9      qDebug() << "[GameEngine] 时间缩放设置为:" << m_timeScale;
10 }
11
12 float GameEngine::getFPS() const { return m_fps; }
13 float GameEngine::getAverageFPS() const { return m_averageFPS; }
14 float GameEngine::getGameTime() const { return m_gameTime; }
15 bool GameEngine::isPaused() const { return m_isPaused; }
16
17 void GameEngine::setPaused(bool paused)
18 {
19     if (m_isPaused != paused) {
20         m_isPaused = paused;
21         qDebug() << "[GameEngine] 游戏" << (paused ? "暂停" : "继续");
22         emit pauseStateChanged(paused);
23     }
24 }
```

注意：需要在gameEngine.h的signals区域添加 `void pauseStateChanged(bool paused);` 信号声明。

1.4 析构函数修改

别忘了在析构函数中释放m_frameTimer:

gameEngine.cpp - 析构函数新增

```
1  // 在析构函数中添加:
2  if (m_frameTimer) {
3      delete m_frameTimer;
4      m_frameTimer = nullptr;
```

```
5 }
```

1.5 验证测试

1. 点击左下角锤子图标（构建项目），确保编译通过
2. 点击运行按钮，启动游戏
3. 查看控制台输出，确认没有错误
4. 可以在update()中添加调试输出，验证FPS和deltaTime

可选：添加FPS调试输出（每秒输出一次）：

FPS调试输出示例

```
1 // 在update()方法末尾添加：
2 static float fpsDebugTimer = 0.0f;
3 fpsDebugTimer += m_unscaledDeltaTime;
4 if (fpsDebugTimer >= 1.0f) {
5     qDebug() << QString("[GameEngine] FPS: %1 平均FPS: %2 DeltaTime: %3ms")
6         .arg(m_fps, 0, 'f', 1)
7         .arg(m_averageFPS, 0, 'f', 1)
8         .arg(m_deltaTime * 1000, 0, 'f', 2);
9     fpsDebugTimer = 0.0f;
10 }
```

验收标准

- ☐ 编译无错误
- ☐ 程序能正常运行
- ☐ deltaTime计算正确（约16.67ms@60FPS）
- ☐ FPS计数正常工作
- ☐ 时间缩放功能正常（setTimeScale）
- ☐ 暂停功能正常（setPaused）
- ☐ 固定时间步长逻辑正确

三、步骤2：实现事件系统



目标：实现事件驱动架构，建立事件总线，支持事件的订阅与发布，实现各系统间的解耦通信

新增文件：eventManager.h / eventManager.cpp、gameEvent.h

重要性：★★★★★（核心架构，后续所有系统都依赖它）

2.1 为什么需要事件系统

事件驱动架构是游戏引擎的核心设计模式之一，它的优势：

- **解耦**：发送者和接收者互不依赖，降低系统间耦合度
- **灵活**：可以动态订阅/取消订阅事件
- **可扩展**：新增事件类型不影响现有代码
- **调试方便**：可以统一监控和记录所有事件

2.2 创建事件类型定义文件

操作步骤

1. 在Qt Creator项目树中，右键点击 `src/core/` 目录
2. 选择「Add New...」→「C++」→「C++ Header File」
3. 文件名填写 `gameEvent`，点击下一步
4. 确认路径在src/core/下，点击完成
5. 打开gameEvent.h，输入以下代码
6. 保存文件（Ctrl+S）

gameEvent.h 完整代码

```
gameEvent.h

1  #ifndef GAMEEVENT_H
2  #define GAMEEVENT_H
3
4  #include <QString>;
5  #include <QVariant>;
6  #include <QMap>;
7  #include <QPointF>;
8
9  /**
10   * @enum EventType
11   * @brief 游戏事件类型枚举
12   *
13   * 所有游戏事件的类型定义，按系统分类
14   */
15  enum class EventType {
```

```
16 // 游戏事件
17 GameStart, // 游戏开始
18 GamePause, // 游戏暂停
19 GameResume, // 游戏继续
20 GameOver, // 游戏结束
21 GameRestart, // 游戏重启
22
23 // 场景事件
24 SceneChanging, // 场景切换中
25 SceneChanged, // 场景切换完成
26 SceneLoaded, // 场景加载完成
27
28 // 输入事件
29 KeyPressed, // 按键按下（瞬时）
30 KeyReleased, // 按键释放
31 MouseMoved, // 鼠标移动
32 MouseButtonPressed, // 鼠标按钮按下
33 MouseButtonReleased, // 鼠标按钮释放
34 MouseWheel, // 鼠标滚轮
35 ActionTriggered, // 输入动作触发
36
37 // 玩家事件
38 PlayerSpawned, // 玩家生成
39 PlayerDied, // 玩家死亡
40 PlayerDamaged, // 玩家受伤
41 PlayerHealed, // 玩家治疗
42 PlayerLevelUp, // 玩家升级
43 PlayerMoved, // 玩家移动
44
45 // 敌人事件
46 EnemySpawned, // 敌人生成
47 EnemyDied, // 敌人死亡
48 EnemyDamaged, // 敌人受伤
49
50 // 战斗事件
51 CombatStarted, // 战斗开始
52 CombatEnded, // 战斗结束
53 DamageDealt, // 造成伤害
54 CriticalHit, // 暴击
55
56 // 技能事件
57 SkillUsed, // 技能使用
58 SkillCooldownStart, // 技能冷却开始
59 SkillCooldownEnd, // 技能冷却结束
60 SkillUnlocked, // 技能解锁
61
62 // 资源事件
```

```

63     ResourceLoaded,      // 资源加载完成
64     ResourceLoadFailed, // 资源加载失败
65     ResourceUnloaded,    // 资源卸载
66
67     // 音频事件
68     MusicStarted,        // 音乐开始
69     MusicStopped,        // 音乐停止
70     SoundPlayed,         // 音效播放
71     VolumeChanged,       // 音量变化
72
73     // 配置事件
74     ConfigChanged,       // 配置变更
75
76     // 存档事件
77     SaveCompleted,       // 存档完成
78     LoadCompleted,      // 读档完成
79     AutoSaveTriggered,   // 自动存档触发
80
81     // UI事件
82     UIOpened,            // UI打开
83     UIClosed,            // UI关闭
84     ButtonClicked,       // 按钮点击
85
86     // 自定义事件（用于扩展）
87     Custom                // 自定义事件
88 };
89
90 /**
91  * @class GameEvent
92  * @brief 游戏事件基类
93  *
94  * 所有游戏事件的基类，包含事件类型和事件数据
95  */
96 class GameEvent
97 {
98 public:
99     /**
100      * @brief 构造函数
101      * @param type 事件类型
102      */
103     explicit GameEvent(EventType type)
104         : m_type(type)
105         , m_propagationStopped(false)
106     {
107     }
108
109     virtual ~GameEvent() = default;

```

```
110
111     /**
112     * @brief 获取事件类型
113     * @return 事件类型
114     */
115     EventType getType() const { return m_type; }
116
117     /**
118     * @brief 获取事件类型名称（用于调试）
119     * @return 事件类型名称字符串
120     */
121     QString getTypeName() const
122     {
123         return getEventTypeName(m_type);
124     }
125
126     /**
127     * @brief 设置事件数据
128     * @param key 数据键
129     * @param value 数据值
130     */
131     void setData(const QString& key, const QVariant& value)
132     {
133         m_data[key] = value;
134     }
135
136     /**
137     * @brief 获取事件数据
138     * @param key 数据键
139     * @param defaultValue 默认值
140     * @return 数据值
141     */
142     QVariant getData(const QString& key, const QVariant& defaultValue =
143     QVariant()) const
144     {
145         return m_data.value(key, defaultValue);
146     }
147
148     /**
149     * @brief 检查是否有指定数据
150     * @param key 数据键
151     * @return 是否存在
152     */
153     bool hasData(const QString& key) const
154     {
155         return m_data.contains(key);
156     }
```

```
156
157     /**
158     * @brief 获取所有数据
159     * @return 数据映射
160     */
161     const QMap<QString, QVariant>& getAllData() const { return m_data; }
162
163     /**
164     * @brief 停止事件传播
165     *
166     * 调用后，后续的监听器将不会收到此事件
167     */
168     void stopPropagation() { m_propagationStopped = true; }
169
170     /**
171     * @brief 检查事件传播是否已停止
172     * @return 是否已停止
173     */
174     bool isPropagationStopped() const { return m_propagationStopped; }
175
176     /**
177     * @brief 获取事件类型名称（静态方法）
178     * @param type 事件类型
179     * @return 类型名称
180     */
181     static QString getEventTypeName(EventType type)
182     {
183         switch (type) {
184             case EventType::GameStart: return "GameStart";
185             case EventType::GamePause: return "GamePause";
186             case EventType::GameResume: return "GameResume";
187             case EventType::GameOver: return "GameOver";
188             case EventType::GameRestart: return "GameRestart";
189             case EventType::SceneChanging: return "SceneChanging";
190             case EventType::SceneChanged: return "SceneChanged";
191             case EventType::SceneLoaded: return "SceneLoaded";
192             case EventType::KeyPressed: return "KeyPressed";
193             case EventType::KeyReleased: return "KeyReleased";
194             case EventType::MouseMoved: return "MouseMoved";
195             case EventType::MouseButtonPressed: return "MouseButtonPressed";
196             case EventType::MouseButtonReleased: return "MouseButtonReleased";
197             case EventType::MouseWheel: return "MouseWheel";
198             case EventType::ActionTriggered: return "ActionTriggered";
199             case EventType::PlayerSpawned: return "PlayerSpawned";
200             case EventType::PlayerDied: return "PlayerDied";
201             case EventType::PlayerDamaged: return "PlayerDamaged";
202             case EventType::PlayerHealed: return "PlayerHealed";
```

```

203         case EventType::PlayerLevelUp: return "PlayerLevelUp";
204         case EventType::PlayerMoved: return "PlayerMoved";
205         case EventType::EnemySpawned: return "EnemySpawned";
206         case EventType::EnemyDied: return "EnemyDied";
207         case EventType::EnemyDamaged: return "EnemyDamaged";
208         case EventType::CombatStarted: return "CombatStarted";
209         case EventType::CombatEnded: return "CombatEnded";
210         case EventType::DamageDealt: return "DamageDealt";
211         case EventType::CriticalHit: return "CriticalHit";
212         case EventType::SkillUsed: return "SkillUsed";
213         case EventType::SkillCooldownStart: return "SkillCooldownStart";
214         case EventType::SkillCooldownEnd: return "SkillCooldownEnd";
215         case EventType::SkillUnlocked: return "SkillUnlocked";
216         case EventType::ResourceLoaded: return "ResourceLoaded";
217         case EventType::ResourceLoadFailed: return "ResourceLoadFailed";
218         case EventType::ResourceUnloaded: return "ResourceUnloaded";
219         case EventType::MusicStarted: return "MusicStarted";
220         case EventType::MusicStopped: return "MusicStopped";
221         case EventType::SoundPlayed: return "SoundPlayed";
222         case EventType::VolumeChanged: return "VolumeChanged";
223         case EventType::ConfigChanged: return "ConfigChanged";
224         case EventType::SaveCompleted: return "SaveCompleted";
225         case EventType::LoadCompleted: return "LoadCompleted";
226         case EventType::AutoSaveTriggered: return "AutoSaveTriggered";
227         case EventType::UIOpened: return "UIOpened";
228         case EventType::UIClosed: return "UIClosed";
229         case EventType::ButtonClicked: return "ButtonClicked";
230         case EventType::Custom: return "Custom";
231         default: return "Unknown";
232     }
233 }
234
235 private:
236     EventType m_type;                ///< 事件类型
237     QMap<QString, QVariant> m_data;    ///< 事件数据
238     bool m_propagationStopped;        ///< 事件传播是否停止
239 };
240
241 /**
242  * @typedef EventListener
243  * @brief 事件监听器函数类型
244  *
245  * 接收GameEvent指针的函数，可以是普通函数、lambda或成员函数（使用std::bind）
246  */
247 #include <functional>;
248 using EventListener = std::function<void(GameEvent*)>;
249

```

```
250  /**
251   * @typedef EventListenerId
252   * @brief 事件监听器ID类型
253   */
254  using EventListenerId = int;
255
256  #endif // GAMEEVENT_H
```

2.3 创建事件管理器

操作步骤

1. 在Qt Creator项目树中，右键点击 `src/core/` 目录
2. 选择「Add New...」→「C++」→「C++ Class」
3. 类名填写 `EventManager`，基类选择QObject
4. 点击下一步，确认路径在src/core/下，点击完成
5. 分别打开eventManager.h和eventManager.cpp，输入代码
6. 保存所有文件（Ctrl+Shift+S）

eventManager.h 完整代码

```
eventManager.h

1  #ifndef EVENTMANAGER_H
2  #define EVENTMANAGER_H
3
4  #include <QObject>;
5  #include <QMap>;
6  #include <QList>;
7  #include <memory>;
8  #include "gameEvent.h"
9
10 /**
11  * @class EventManager
12  * @brief 事件管理器
13  *
14  * 实现事件总线模式，支持事件的订阅、发布和取消订阅
15  * 所有系统间的通信都应通过事件系统进行，以实现解耦
16  */
17 class EventManager : public QObject
18 {
19     Q_OBJECT
20 public:
```

```

21     explicit EventManager(QObject* parent = nullptr);
22     ~EventManager() override;
23
24     bool initialize();
25
26     /**
27      * @brief 订阅事件
28      * @param type 事件类型
29      * @param listener 监听器函数
30      * @return 监听器ID, 用于取消订阅
31      */
32     EventListenerId subscribe(EventType type, const EventListener& listener);
33
34     /**
35      * @brief 取消订阅事件
36      * @param type 事件类型
37      * @param listenerId 监听器ID
38      * @return 是否成功取消
39      */
40     bool unsubscribe(EventType type, EventListenerId listenerId);
41
42     /**
43      * @brief 取消所有订阅
44      * @param listenerId 监听器ID
45      * @return 取消的订阅数量
46      */
47     int unsubscribeAll(EventListenerId listenerId);
48
49     /**
50      * @brief 发布事件
51      * @param event 事件对象
52      *
53      * 事件会按订阅顺序依次发送给所有监听器
54      * 如果事件调用了stopPropagation(), 后续监听器将不会收到
55      */
56     void publish(GameEvent* event);
57
58     /**
59      * @brief 发布事件 (便捷方法)
60      * @param type 事件类型
61      * @param data 事件数据 (可选)
62      */
63     void publishEvent(EventType type, const QMap<QString, QVariant>&
data = {});
64
65     /**
66      * @brief 检查是否有订阅者

```



```

67     * @param type 事件类型
68     * @return 是否有订阅者
69     */
70     bool hasListeners(EventType type) const;
71
72     /**
73     * @brief 获取订阅者数量
74     * @param type 事件类型
75     * @return 订阅者数量
76     */
77     int getListenerCount(EventType type) const;
78
79     /**
80     * @brief 清空所有订阅
81     */
82     void clearAllListeners();
83
84     /**
85     * @brief 启用/禁用事件调试输出
86     * @param enabled 是否启用
87     */
88     void setDebugEnabled(bool enabled) { m_debugEnabled = enabled; }
89
90     /**
91     * @brief 检查调试输出是否启用
92     * @return 是否启用
93     */
94     bool isDebugEnabled() const { return m_debugEnabled; }
95
96     signals:
97     /**
98     * @brief 事件发布信号（用于Qt信号槽机制）
99     * @param type 事件类型
100    * @param event 事件对象
101    */
102    void eventPublished(EventType type, GameEvent* event);
103
104    private:
105        struct ListenerInfo {
106            EventListenerId id;
107            EventListener listener;
108        };
109
110        QMap<EventType, QList<ListenerInfo>> m_listeners; ///< 事件监听
器映射
111        EventListenerId m_nextListenerId;                ///< 下一个监听器ID
112        bool m_debugEnabled;                             ///< 是否启用调试输出

```

```

113     bool m_isPublishing;                                     ///< 是否正在发布事件
    (防止递归)
114
115     /**
116      * @brief 生成唯一的监听器ID
117      * @return 监听器ID
118      */
119     EventListenerId generateListenerId();
120 };
121
122 #endif // EVENTMANAGER_H

```

eventManager.cpp 完整代码

```

eventManager.cpp

1  #include "eventManager.h"
2  #include <QDebug>;
3
4  EventManager::EventManager(QObject* parent)
5      : QObject(parent)
6      , m_listeners()
7      , m_nextListenerId(1)
8      , m_debugEnabled(false)
9      , m_isPublishing(false)
10 {
11 }
12
13 EventManager::~EventManager()
14 {
15     clearAllListeners();
16 }
17
18 bool EventManager::initialize()
19 {
20     qDebug() << "[EventManager] 初始化事件管理器...";
21     m_nextListenerId = 1;
22     qDebug() << "[EventManager] 事件管理器初始化完成。";
23     return true;
24 }
25
26 EventListenerId EventManager::subscribe(EventType type, const EventListener&
listener)
27 {
28     EventListenerId id = generateListenerId();
29

```

```

30     ListenerInfo info;
31     info.id = id;
32     info.listener = listener;
33
34     m_listeners[type].append(info);
35
36     if (m_debugEnabled) {
37         qDebug() << "[EventManager] 订阅事件:" <<
GameEvent::getEventTypeName(type)
38             << "ID:" << id;
39     }
40
41     return id;
42 }
43
44 bool EventManager::unsubscribe(EventType type, EventListenerId listenerId)
45 {
46     if (!m_listeners.contains(type)) {
47         return false;
48     }
49
50     auto& listeners = m_listeners[type];
51     for (int i = 0; i < listeners.size(); ++i) {
52         if (listeners[i].id == listenerId) {
53             listeners.removeAt(i);
54             if (m_debugEnabled) {
55                 qDebug() << "[EventManager] 取消订阅:" <<
GameEvent::getEventTypeName(type)
56                     << "ID:" << listenerId;
57             }
58             return true;
59         }
60     }
61
62     return false;
63 }
64
65 int EventManager::unsubscribeAll(EventListenerId listenerId)
66 {
67     int count = 0;
68     for (auto it = m_listeners.begin(); it != m_listeners.end(); ++it) {
69         auto& listeners = it.value();
70         for (int i = listeners.size() - 1; i >= 0; --i) {
71             if (listeners[i].id == listenerId) {
72                 listeners.removeAt(i);
73                 count++;
74             }

```

```
75     }
76 }
77
78 if (m_debugEnabled && count > 0) {
79     qDebug() << "[EventManager] 取消所有订阅, ID:" << listenerId
80         << "共取消" << count << "个";
81 }
82
83 return count;
84 }
85
86 void EventManager::publish(GameEvent* event)
87 {
88     if (!event) {
89         return;
90     }
91
92     if (m_isPublishing) {
93         qWarning() << "[EventManager] 检测到递归事件发布, 已忽略:"
94             << event->getTypeName();
95         return;
96     }
97
98     m_isPublishing = true;
99
100     EventType type = event->getType();
101
102     if (m_debugEnabled) {
103         qDebug() << "[EventManager] 发布事件:" << event->getTypeName();
104     }
105
106     // 发送Qt信号
107     emit eventPublished(type, event);
108
109     // 调用所有监听器
110     if (m_listeners.contains(type)) {
111         const auto& listeners = m_listeners[type];
112         for (const auto& info : listeners) {
113             if (event->isPropagationStopped()) {
114                 break;
115             }
116             if (info.listener) {
117                 info.listener(event);
118             }
119         }
120     }
121 }
```

```

122     m_isPublishing = false;
123 }
124
125 void EventManager::publishEvent(EventType type, const QMap<QString,
    QVariant>& data)
126 {
127     GameEvent event(type);
128     for (auto it = data.constBegin(); it != data.constEnd(); ++it) {
129         event.setData(it.key(), it.value());
130     }
131     publish(&event);
132 }
133
134 bool EventManager::hasListeners(EventType type) const
135 {
136     return m_listeners.contains(type) && !m_listeners[type].isEmpty();
137 }
138
139 int EventManager::getListenerCount(EventType type) const
140 {
141     if (!m_listeners.contains(type)) {
142         return 0;
143     }
144     return m_listeners[type].size();
145 }
146
147 void EventManager::clearAllListeners()
148 {
149     m_listeners.clear();
150     m_nextListenerId = 1;
151     qDebug() << "[EventManager] 已清空所有事件监听器。";
152 }
153
154 EventListenerId EventManager::generateListenerId()
155 {
156     return m_nextListenerId++;
157 }

```

2.4 将EventManager集成到GameEngine

操作步骤

1. 打开 `src/core/gameEngine.h`
2. 添加EventManager的前向声明和成员变量
3. 添加getManager()方法

4. 保存文件
5. 打开 `src/core/gameEngine.cpp`
6. 添加eventManager.h头文件引用
7. 在initializeSubsystems()中初始化EventManager
8. 在shutdownSubsystems()中销毁EventManager
9. 添加getEventManager()实现
10. 保存文件

gameEngine.h 修改

gameEngine.h - 添加EventManager

```
1 // 前向声明区域添加:
2 class EventManager;
3
4 // 成员变量区域添加:
5 std::unique_ptr<EventManager> m_eventManager;
6
7 // public方法添加:
8 EventManager* getEventManager() const;
```

gameEngine.cpp 修改

gameEngine.cpp - 集成EventManager

```
1 // 头文件引用添加:
2 #include "eventManager.h"
3
4 // initializeSubsystems()中添加（放在GameConfig之后，ResourceManager之前）:
5 // 初始化事件管理器
6 m_eventManager = std::make_unique<EventManager>();
7 if (!m_eventManager->initialize()) { return false; }
8 qDebug() << "[GameEngine] ✓ 事件管理器初始化完成";
9
10 // shutdownSubsystems()中添加（对应位置）:
11 m_eventManager.reset();
12
13 // 添加getter实现:
14 EventManager* GameEngine::getEventManager() const { return
    m_eventManager.get(); }
```

注意：事件管理器应该在比较早的位置初始化，因为很多其他系统可能会依赖它。

2.5 更新core模块CMakeLists.txt

将EventManager和gameEvent添加到CMakeLists.txt中：

src/core/CMakeLists.txt - 更新

```
1  add_library(core STATIC
2      gameEngine.h
3      gameEngine.cpp
4      sceneManager.h
5      sceneManager.cpp
6      resourceManager.h
7      resourceManager.cpp
8      inputManager.h
9      inputManager.cpp
10     audioManager.h
11     audioManager.cpp
12     gameConfig.h
13     gameConfig.cpp
14     saveSystem.h
15     saveSystem.cpp
16     pluginManager.h
17     pluginManager.cpp
18     eventManager.h      # 新增
19     eventManager.cpp    # 新增
20     gameEvent.h        # 新增
21 )
```

2.6 验证测试

1. 构建项目，确保编译通过
2. 运行游戏，查看控制台输出，确认EventManager初始化成功
3. 可以在GameEngine的start()方法中添加测试代码，验证事件系统

测试代码示例（可选）：

事件系统测试代码

```
1  // 在GameEngine::start()中添加测试：
2  if (m_eventManager) {
3      // 启用调试输出
4      m_eventManager->setDebugEnabled(true);
5
6      // 订阅测试
7      auto testId = m_eventManager->subscribe(EventType::GameStart, []
      (GameEvent* event) {
```

```
8         qDebug() << "[测试] 收到GameStart事件!";
9         qDebug() << "[测试] 事件数据:" << event->getAllData();
10    });
11
12    // 发布测试事件
13    QMap<QString, QVariant> testData;
14    testData["message"] = "Hello, Event System!";
15    testData["value"] = 42;
16    m_eventManager->publishEvent(EventType::GameStart, testData);
17
18    // 取消订阅
19    m_eventManager->unsubscribe(EventType::GameStart, testId);
20 }
```

验收标准

- ☐ 编译无错误
- ☐ EventManager能正常初始化
- ☐ 事件订阅/发布功能正常
- ☐ 事件取消订阅功能正常
- ☐ 事件数据传递正确
- ☐ 事件传播停止功能正常（stopPropagation）
- ☐ EventManager已集成到GameEngine
- ☐ CMakeLists.txt已更新

四、步骤3：完善场景系统



目标：创建场景基类，实现场景生命周期管理，支持场景切换过渡效果

新增文件：scene.h / scene.cpp（场景基类）

修改文件：sceneManager.h / sceneManager.cpp

3.1 为什么需要场景基类

第一阶段的SceneManager只是简单地管理QGraphicsScene对象，缺少游戏场景的生命周期管理。创建场景基类的好处：

- **统一生命周期：**onEnter、onExit、update、render等标准接口

- **资源管理**：场景加载时加载资源，卸载时释放资源
- **切换过渡**：支持淡入淡出等场景切换效果
- **易于扩展**：每种场景（主菜单、游戏、设置等）都继承自基类

3.2 创建场景基类

操作步骤

1. 在Qt Creator项目树中，右键点击 `src/core/` 目录
2. 选择「Add New...」→「C++」→「C++ Class」
3. 类名填写 `Scene`，基类选择QObject
4. 点击下一步，确认路径，点击完成
5. 分别打开scene.h和scene.cpp，输入代码
6. 保存所有文件

scene.h 完整代码

```
scene.h

1  #ifndef SCENE_H
2  #define SCENE_H
3
4  #include <QObject>
5  #include <QGraphicsScene>
6  #include <QString>
7  #include <QPainter>
8
9  class GameEngine;
10
11  /**
12   * @class Scene
13   * @brief 游戏场景基类
14   *
15   * 所有游戏场景的基类，定义了场景的标准生命周期
16   * 每个场景都有自己的QGraphicsScene用于渲染
17   */
18  class Scene : public QObject
19  {
20      Q_OBJECT
21  public:
22      /**
23       * @enum SceneState
24       * @brief 场景状态枚举
```

```

25     */
26     enum SceneState {
27         StateUnloaded,    ///< 未加载
28         StateLoading,     ///< 加载中
29         StateReady,       ///< 就绪（已加载但未激活）
30         StateEntering,    ///< 进入中（过渡动画）
31         StateActive,      ///< 活动中（正常运行）
32         StateExiting,     ///< 退出中（过渡动画）
33         StateError        ///< 错误状态
34     };
35
36     explicit Scene(const QString& name, QObject* parent = nullptr);
37     ~Scene() override;
38
39     // =====
40     // 生命周期方法（子类可重写）
41     // =====
42
43     /**
44      * @brief 加载场景资源
45      *
46      * 加载场景所需的所有资源（图片、音效、配置等）
47      * 耗时操作应该放在这里，避免阻塞游戏循环
48      *
49      * @return 是否加载成功
50      */
51     virtual bool load();
52
53     /**
54      * @brief 卸载场景资源
55      *
56      * 释放场景占用的所有资源
57      */
58     virtual void unload();
59
60     /**
61      * @brief 场景进入时调用
62      *
63      * 场景被激活时调用，可以启动动画、播放音乐等
64      */
65     virtual void onEnter();
66
67     /**
68      * @brief 场景退出时调用
69      *
70      * 场景被停用时调用，可以暂停动画、停止音乐等
71      */

```

```
72     virtual void onExit();
73
74     /**
75      * @brief 更新场景逻辑
76      * @param deltaTime 帧时间 (秒)
77      */
78     virtual void update(float deltaTime);
79
80     /**
81      * @brief 渲染场景 (如有额外渲染需求)
82      * @param painter 画家对象
83      *
84      * 大部分渲染由QGraphicsScene自动处理
85      * 此方法用于需要自定义绘制的内容
86      */
87     virtual void render(QPainter* painter);
88
89     /**
90      * @brief 调整大小
91      * @param width 新宽度
92      * @param height 新高度
93      */
94     virtual void resize(int width, int height);
95
96     // =====
97     // 属性方法
98     // =====
99
100    /**
101     * @brief 获取场景名称
102     * @return 场景名称
103     */
104    QString getName() const { return m_name; }
105
106    /**
107     * @brief 获取场景状态
108     * @return 场景状态
109     */
110    SceneState getState() const { return m_state; }
111
112    /**
113     * @brief 获取QGraphicsScene对象
114     * @return QGraphicsScene指针
115     */
116    QGraphicsScene* getGraphicsScene() const { return m_graphicsScene; }
117
118    /**
```

```
119     * @brief 检查场景是否已加载
120     * @return 是否已加载
121     */
122     bool isLoading() const { return m_state >= StateReady; }
123
124     /**
125     * @brief 检查场景是否处于活动状态
126     * @return 是否活动
127     */
128     bool isActive() const { return m_state == StateActive; }
129
130     /**
131     * @brief 获取游戏引擎指针
132     * @return 游戏引擎指针
133     */
134     GameEngine* getGameEngine() const { return m_gameEngine; }
135
136     /**
137     * @brief 设置游戏引擎指针
138     * @param engine 游戏引擎指针
139     */
140     void setGameEngine(GameEngine* engine) { m_gameEngine = engine; }
141
142     // =====
143     // 过渡效果
144     // =====
145
146     /**
147     * @brief 开始进入过渡
148     * @param duration 过渡时长 (秒)
149     */
150     void startEnterTransition(float duration = 0.5f);
151
152     /**
153     * @brief 开始退出过渡
154     * @param duration 过渡时长 (秒)
155     */
156     void startExitTransition(float duration = 0.5f);
157
158     /**
159     * @brief 获取过渡进度 (0.0 - 1.0)
160     * @return 过渡进度
161     */
162     float getTransitionProgress() const { return m_transitionProgress; }
163
164     /**
165     * @brief 检查是否正在过渡
```

```

166     * @return 是否正在过渡
167     */
168     bool isTransitioning() const { return m_state == StateEntering || m_state
== StateExiting; }
169
170 signals:
171     /**
172     * @brief 场景加载完成信号
173     */
174     void loaded();
175
176     /**
177     * @brief 场景卸载信号
178     */
179     void unloaded();
180
181     /**
182     * @brief 场景进入完成信号
183     */
184     void enterFinished();
185
186     /**
187     * @brief 场景退出完成信号
188     */
189     void exitFinished();
190
191     /**
192     * @brief 场景状态变化信号
193     * @param newState 新状态
194     */
195     void stateChanged(SceneState newState);
196
197 protected:
198     /**
199     * @brief 设置场景状态
200     * @param state 新状态
201     */
202     void setState(SceneState state);
203
204     /**
205     * @brief 创建QGraphicsScene
206     *
207     * 子类可以重写此方法来创建自定义的QGraphicsScene
208     */
209     virtual void createGraphicsScene();
210
211     QString m_name;                ///< 场景名称

```

```

212     SceneState m_state;          ///< 场景状态
213     QGraphicsScene* m_graphicsScene; ///< Qt图形场景
214     GameEngine* m_gameEngine;    ///< 游戏引擎指针
215
216     // 过渡效果
217     float m_transitionProgress;   ///< 过渡进度 (0.0 - 1.0)
218     float m_transitionDuration;   ///< 过渡时长 (秒)
219     bool m_transitionDirection;   ///< 过渡方向 (true=进入, false=退出)
220
221 private:
222     /**
223      * @brief 更新过渡效果
224      * @param deltaTime 帧时间
225      */
226     void updateTransition(float deltaTime);
227 };
228
229 #endif // SCENE_H

```

scene.cpp 完整代码

```

scene.cpp

1  #include "scene.h"
2  #include <QDebug>;
3  #include <QGraphicsScene>;
4
5  Scene::Scene(const QString& name, QObject* parent)
6      : QObject(parent)
7      , m_name(name)
8      , m_state(StateUnloaded)
9      , m_graphicsScene(nullptr)
10     , m_gameEngine(nullptr)
11     , m_transitionProgress(0.0f)
12     , m_transitionDuration(0.5f)
13     , m_transitionDirection(true)
14 {
15 }
16
17 Scene::~Scene()
18 {
19     if (m_state != StateUnloaded) {
20         unload();
21     }
22     if (m_graphicsScene) {
23         delete m_graphicsScene;

```

```
24         m_graphicsScene = nullptr;
25     }
26 }
27
28 bool Scene::load()
29 {
30     if (m_state != StateUnloaded) {
31         qWarning() << "[Scene] 场景已加载:" << m_name;
32         return true;
33     }
34
35     qDebug() << "[Scene] 加载场景:" << m_name;
36     setState(StateLoading);
37
38     // 创建图形场景
39     createGraphicsScene();
40
41     if (!m_graphicsScene) {
42         qCritical() << "[Scene] 创建图形场景失败:" << m_name;
43         setState(StateError);
44         return false;
45     }
46
47     // TODO: 子类在这里加载具体资源
48
49     setState(StateReady);
50     emit loaded();
51
52     qDebug() << "[Scene] 场景加载完成:" << m_name;
53     return true;
54 }
55
56 void Scene::unload()
57 {
58     if (m_state == StateUnloaded) {
59         return;
60     }
61
62     qDebug() << "[Scene] 卸载场景:" << m_name;
63
64     // 如果场景是活动的, 先退出
65     if (m_state == StateActive || m_state == StateEntering) {
66         onExit();
67     }
68
69     // TODO: 子类在这里释放具体资源
70 }
```

```
71     // 销毁图形场景
72     if (m_graphicsScene) {
73         delete m_graphicsScene;
74         m_graphicsScene = nullptr;
75     }
76
77     setState(StateUnloaded);
78     emit unloaded();
79
80     qDebug() << "[Scene] 场景已卸载:" << m_name;
81 }
82
83 void Scene::onEnter()
84 {
85     qDebug() << "[Scene] 进入场景:" << m_name;
86     // TODO: 子类在这里实现进入逻辑
87     // 例如: 播放背景音乐、启动动画等
88 }
89
90 void Scene::onExit()
91 {
92     qDebug() << "[Scene] 退出场景:" << m_name;
93     // TODO: 子类在这里实现退出逻辑
94     // 例如: 停止背景音乐、暂停动画等
95 }
96
97 void Scene::update(float deltaTime)
98 {
99     // 更新过渡效果
100     if (isTransitioning()) {
101         updateTransition(deltaTime);
102     }
103
104     // TODO: 子类在这里实现具体的更新逻辑
105 }
106
107 void Scene::render(QPainter* painter)
108 {
109     Q_UNUSED(painter);
110     // TODO: 子类在这里实现自定义渲染
111     // 大部分渲染由QGraphicsScene自动处理
112 }
113
114 void Scene::resize(int width, int height)
115 {
116     Q_UNUSED(width);
117     Q_UNUSED(height);
```



```
118     // TODO: 子类在这里处理窗口大小变化
119 }
120
121 void Scene::startEnterTransition(float duration)
122 {
123     if (m_state != StateReady && m_state != StateActive) {
124         qWarning() << "[Scene] 无法开始进入过渡, 状态不正确:" << m_name;
125         return;
126     }
127
128     m_transitionDuration = duration;
129     m_transitionProgress = 0.0f;
130     m_transitionDirection = true;
131     setState(StateEntering);
132
133     qDebug() << "[Scene] 开始进入过渡:" << m_name << "时长:" << duration << "秒";
134 }
135
136 void Scene::startExitTransition(float duration)
137 {
138     if (m_state != StateActive) {
139         qWarning() << "[Scene] 无法开始退出过渡, 状态不正确:" << m_name;
140         return;
141     }
142
143     m_transitionDuration = duration;
144     m_transitionProgress = 0.0f;
145     m_transitionDirection = false;
146     setState(StateExiting);
147
148     qDebug() << "[Scene] 开始退出过渡:" << m_name << "时长:" << duration << "秒";
149 }
150
151 void Scene::setState(SceneState state)
152 {
153     if (m_state != state) {
154         m_state = state;
155         emit stateChanged(m_state);
156     }
157 }
158
159 void Scene::createGraphicsScene()
160 {
161     m_graphicsScene = new QGraphicsScene();
162     m_graphicsScene->setSceneRect(0, 0, 1280, 720);
163     m_graphicsScene->setBackgroundBrush(QBrush(QColor(20, 20, 40)));
164 }
```

```

165
166 void Scene::updateTransition(float deltaTime)
167 {
168     if (m_transitionDuration <= 0.0f) {
169         // 无过渡，直接完成
170         m_transitionProgress = 1.0f;
171     } else {
172         m_transitionProgress += deltaTime / m_transitionDuration;
173         m_transitionProgress = qMin(m_transitionProgress, 1.0f);
174     }
175
176     if (m_transitionProgress >= 1.0f) {
177         // 过渡完成
178         if (m_transitionDirection) {
179             // 进入过渡完成
180             setState(StateActive);
181             onEnter();
182             emit enterFinished();
183             qDebug() << "[Scene] 进入过渡完成:" << m_name;
184         } else {
185             // 退出过渡完成
186             setState(StateReady);
187             onExit();
188             emit exitFinished();
189             qDebug() << "[Scene] 退出过渡完成:" << m_name;
190         }
191     }
192 }

```

3.3 修改SceneManager支持Scene基类

操作步骤

1. 打开 `src/core/sceneManager.h`
2. 添加scene.h头文件引用
3. 修改场景存储类型，从QGraphicsScene*改为Scene*
4. 添加场景栈、过渡管理等新功能
5. 保存文件
6. 打开 `src/core/sceneManager.cpp`
7. 重写所有方法以支持Scene基类
8. 添加场景切换过渡逻辑

9. 保存文件

sceneManager.h 修改后完整代码

sceneManager.h - 完整更新版

```
1  #ifndef SCENEMANAGER_H
2  #define SCENEMANAGER_H
3
4  #include <QObject>;
5  #include <QGraphicsView>;
6  #include <QStack>;
7  #include <QMap>;
8  #include <QString>;
9  #include "scene.h"
10
11  /**
12   * @class SceneManager
13   * @brief 场景管理器
14   *
15   * 管理游戏中的所有场景，负责场景的注册、切换、生命周期管理
16   * 支持场景栈（用于暂停菜单等叠加场景）和场景切换过渡
17   */
18  class SceneManager : public QObject
19  {
20      Q_OBJECT
21  public:
22      explicit SceneManager(QObject* parent = nullptr);
23      ~SceneManager() override;
24
25      bool initialize();
26
27      // =====
28      // 场景注册
29      // =====
30
31      /**
32       * @brief 注册场景
33       * @param sceneName 场景名称
34       * @param scene 场景对象
35       * @return 是否注册成功
36       */
37      bool registerScene(const QString& sceneName, Scene* scene);
38
39      /**
40       * @brief 移除场景
41       * @param sceneName 场景名称
```

```
42     * @return 是否移除成功
43     */
44     bool removeScene(const QString& sceneName);
45
46     /**
47     * @brief 检查场景是否存在
48     * @param sceneName 场景名称
49     * @return 是否存在
50     */
51     bool hasScene(const QString& sceneName) const;
52
53     /**
54     * @brief 获取场景对象
55     * @param sceneName 场景名称
56     * @return 场景指针
57     */
58     Scene* getScene(const QString& sceneName) const;
59
60     // =====
61     // 场景切换
62     // =====
63
64     /**
65     * @brief 切换到指定场景
66     * @param sceneName 场景名称
67     * @param useTransition 是否使用过渡效果
68     * @return 是否切换成功
69     */
70     bool switchToScene(const QString& sceneName, bool useTransition = true);
71
72     /**
73     * @brief 推送场景到栈顶（叠加场景）
74     * @param sceneName 场景名称
75     * @param useTransition 是否使用过渡效果
76     * @return 是否成功
77     *
78     * 用于暂停菜单、对话框等需要叠加显示的场景
79     * 当前场景会被暂停但不会卸载
80     */
81     bool pushScene(const QString& sceneName, bool useTransition = true);
82
83     /**
84     * @brief 弹出栈顶场景
85     * @param useTransition 是否使用过渡效果
86     * @return 是否成功
87     */
88     bool popScene(bool useTransition = true);
```

```
89
90     /**
91      * @brief 清空场景栈
92      */
93     void clearSceneStack();
94
95     // =====
96     // 当前场景
97     // =====
98
99     /**
100     * @brief 获取当前场景
101     * @return 当前场景指针
102     */
103     Scene* getCurrentScene() const;
104
105     /**
106     * @brief 获取当前场景名称
107     * @return 当前场景名称
108     */
109     QString getCurrentSceneName() const;
110
111     /**
112     * @brief 获取场景栈深度
113     * @return 栈深度
114     */
115     int getSceneStackSize() const;
116
117     /**
118     * @brief 检查是否正在切换场景
119     * @return 是否正在切换
120     */
121     bool isTransitioning() const;
122
123     // =====
124     // 视图管理
125     // =====
126
127     /**
128     * @brief 获取视图对象
129     * @return QGraphicsView指针
130     */
131     QGraphicsView* getView() const;
132
133     /**
134     * @brief 设置视图大小
135     * @param width 宽度
```

```

136     * @param height 高度
137     */
138     void setViewSize(int width, int height);
139
140     // =====
141     // 更新
142     // =====
143
144     /**
145     * @brief 更新当前场景
146     * @param deltaTime 帧时间 (秒)
147     */
148     void updateCurrentScene(float deltaTime);
149
150     signals:
151     /**
152     * @brief 场景切换开始信号
153     * @param fromScene 源场景名称
154     * @param toScene 目标场景名称
155     */
156     void sceneChanging(const QString& fromScene, const QString& toScene);
157
158     /**
159     * @brief 场景切换完成信号
160     * @param fromScene 源场景名称
161     * @param toScene 目标场景名称
162     */
163     void sceneChanged(const QString& fromScene, const QString& toScene);
164
165     /**
166     * @brief 场景入栈信号
167     * @param sceneName 场景名称
168     */
169     void scenePushed(const QString& sceneName);
170
171     /**
172     * @brief 场景出栈信号
173     * @param sceneName 场景名称
174     */
175     void scenePopped(const QString& sceneName);
176
177     private:
178     QGraphicsView* m_view;                ///< 视图对象
179     QMap<QString, Scene*> m_scenes;        ///< 已注册的场景映射
180     QStack<Scene*> m_sceneStack;          ///< 场景栈
181     Scene* m_currentScene;                ///< 当前活动场景
182     QString m_currentSceneName;           ///< 当前场景名称

```

```

183
184     // 切换过渡
185     bool m_isTransitioning;           ///< 是否正在过渡
186     QString m_pendingSceneName;      ///< 待切换的场景名称
187     bool m_pendingIsPush;            ///< 待切换是否是push操作
188     bool m_useTransition;             ///< 是否使用过渡效果
189     static constexpr float DEFAULT_TRANSITION_DURATION = 0.5f; ///< 默认过渡时长
190
191     /**
192      * @brief 执行场景切换
193      * @param sceneName 目标场景名称
194      * @param isPush 是否是push操作
195      * @param useTransition 是否使用过渡
196      * @return 是否成功
197      */
198     bool doSceneChange(const QString& sceneName, bool isPush, bool
useTransition);
199
200     /**
201      * @brief 完成场景切换
202      */
203     void finishSceneChange();
204
205     /**
206      * @brief 创建视图
207      */
208     void createView();
209 };
210
211 #endif // SCENEMANAGER_H

```

sceneManager.cpp 修改后完整代码

sceneManager.cpp - 完整更新版

```

1  #include "sceneManager.h"
2  #include <QDebug>;
3  #include <QGraphicsView>;
4  #include <QGraphicsScene>;
5
6  SceneManager::SceneManager(QObject* parent)
7      : QObject(parent)
8      , m_view(nullptr)
9      , m_scenes()
10     , m_sceneStack()
11     , m_currentScene(nullptr)

```

```
12     , m_currentSceneName()
13     , m_isTransitioning(false)
14     , m_pendingSceneName()
15     , m_pendingIsPush(false)
16     , m_useTransition(true)
17 {
18 }
19
20 SceneManager::~SceneManager()
21 {
22     clearSceneStack();
23
24     // 删除所有已注册的场景
25     qDeleteAll(m_scenes);
26     m_scenes.clear();
27
28     if (m_view) {
29         delete m_view;
30         m_view = nullptr;
31     }
32 }
33
34 bool SceneManager::initialize()
35 {
36     qDebug() << "[SceneManager] 初始化场景管理器...";
37
38     // 创建视图
39     createView();
40
41     if (!m_view) {
42         qCritical() << "[SceneManager] 创建视图失败!";
43         return false;
44     }
45
46     qDebug() << "[SceneManager] 场景管理器初始化完成。";
47     return true;
48 }
49
50 void SceneManager::createView()
51 {
52     m_view = new QGraphicsView();
53     m_view->setRenderHint(QPainter::Antialiasing, true);
54     m_view->setRenderHint(QPainter::SmoothPixmapTransform, true);
55     m_view->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
56     m_view->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
57     m_view->setViewportUpdateMode(QGraphicsView::FullViewportUpdate);
```



```
58     m_view->setOptimizationFlag(QGraphicsView::DontAdjustForAntialiasing,
59     true);
60 }
61 bool SceneManager::registerScene(const QString& sceneName, Scene* scene)
62 {
63     if (sceneName.isEmpty() || scene == nullptr) {
64         qWarning() << "[SceneManager] 注册场景失败: 无效参数";
65         return false;
66     }
67
68     if (m_scenes.contains(sceneName)) {
69         qWarning() << "[SceneManager] 场景已存在:" << sceneName;
70         return false;
71     }
72
73     m_scenes.insert(sceneName, scene);
74     qDebug() << "[SceneManager] 注册场景:" << sceneName;
75     return true;
76 }
77
78 bool SceneManager::removeScene(const QString& sceneName)
79 {
80     if (!m_scenes.contains(sceneName)) {
81         return false;
82     }
83
84     // 如果是当前场景, 先切换到其他场景
85     if (m_currentSceneName == sceneName) {
86         if (!m_sceneStack.isEmpty()) {
87             popScene(false);
88         } else {
89             m_currentScene = nullptr;
90             m_currentSceneName.clear();
91             m_view->setScene(nullptr);
92         }
93     }
94
95     // 从场景栈中移除
96     for (int i = m_sceneStack.size() - 1; i >= 0; --i) {
97         if (m_sceneStack[i]->getName() == sceneName) {
98             m_sceneStack.remove(i);
99         }
100     }
101
102     Scene* scene = m_scenes.take(sceneName);
103     if (scene) {
```

```
104         scene->unload();
105         delete scene;
106     }
107
108     qDebug() << "[SceneManager] 移除场景:" << sceneName;
109     return true;
110 }
111
112 bool SceneManager::hasScene(const QString& sceneName) const
113 {
114     return m_scenes.contains(sceneName);
115 }
116
117 Scene* SceneManager::getScene(const QString& sceneName) const
118 {
119     return m_scenes.value(sceneName, nullptr);
120 }
121
122 bool SceneManager::switchToScene(const QString& sceneName, bool useTransition)
123 {
124     return doSceneChange(sceneName, false, useTransition);
125 }
126
127 bool SceneManager::pushScene(const QString& sceneName, bool useTransition)
128 {
129     return doSceneChange(sceneName, true, useTransition);
130 }
131
132 bool SceneManager::popScene(bool useTransition)
133 {
134     if (m_sceneStack.isEmpty()) {
135         qWarning() << "[SceneManager] 场景栈为空, 无法弹出";
136         return false;
137     }
138
139     if (m_isTransitioning) {
140         qWarning() << "[SceneManager] 正在过渡中, 无法弹出场景";
141         return false;
142     }
143
144     // 弹出当前场景
145     Scene* oldScene = m_sceneStack.pop();
146     QString oldSceneName = oldScene->getName();
147
148     emit scenePopped(oldSceneName);
149
150     // 切换到栈顶场景
```

```
151     if (!m_sceneStack.isEmpty()) {
152         Scene* newScene = m_sceneStack.top();
153         QString newSceneName = newScene->getName();
154
155         emit sceneChanging(oldSceneName, newSceneName);
156
157         if (useTransition) {
158             // 使用过渡效果
159             m_isTransitioning = true;
160             m_useTransition = true;
161             oldScene->startExitTransition(DEFAULT_TRANSITION_DURATION);
162             // TODO: 等待过渡完成后再切换
163             // 简化处理: 直接切换
164             m_isTransitioning = false;
165         }
166
167         oldScene->onExit();
168         m_currentScene = newScene;
169         m_currentSceneName = newSceneName;
170
171         // 加载场景 (如果未加载)
172         if (!newScene->isLoaded()) {
173             newScene->load();
174         }
175
176         m_view->setScene(newScene->getGraphicsScene());
177         newScene->onEnter();
178
179         emit sceneChanged(oldSceneName, newSceneName);
180         qDebug() << "[SceneManager] 弹出场景, 切换到:" << newSceneName;
181     } else {
182         // 栈空了
183         m_currentScene = nullptr;
184         m_currentSceneName.clear();
185         m_view->setScene(nullptr);
186         oldScene->onExit();
187         qDebug() << "[SceneManager] 弹出场景, 场景栈已空";
188     }
189
190     return true;
191 }
192
193 void SceneManager::clearSceneStack()
194 {
195     // 退出所有场景
196     while (!m_sceneStack.isEmpty()) {
197         Scene* scene = m_sceneStack.pop();
```

```
198         scene->onExit();
199     }
200     m_currentScene = nullptr;
201     m_currentSceneName.clear();
202     if (m_view) {
203         m_view->setScene(nullptr);
204     }
205     qDebug() << "[SceneManager] 场景栈已清空.";
206 }
207
208 Scene* SceneManager::getCurrentScene() const
209 {
210     return m_currentScene;
211 }
212
213 QString SceneManager::getCurrentSceneName() const
214 {
215     return m_currentSceneName;
216 }
217
218 int SceneManager::getSceneStackSize() const
219 {
220     return m_sceneStack.size();
221 }
222
223 bool SceneManager::isTransitioning() const
224 {
225     return m_isTransitioning;
226 }
227
228 QGraphicsView* SceneManager::getView() const
229 {
230     return m_view;
231 }
232
233 void SceneManager::setViewSize(int width, int height)
234 {
235     if (m_view) {
236         m_view->resize(width, height);
237         if (m_currentScene) {
238             m_currentScene->resize(width, height);
239         }
240     }
241 }
242
243 void SceneManager::updateCurrentScene(float deltaTime)
244 {
```

```
245     if (m_currentScene && m_currentScene->isActive()) {
246         m_currentScene->update(deltaTime);
247     }
248 }
249
250 bool SceneManager::doSceneChange(const QString& sceneName, bool isPush, bool
useTransition)
251 {
252     if (!m_scenes.contains(sceneName)) {
253         qWarning() << "[SceneManager] 场景不存在:" << sceneName;
254         return false;
255     }
256
257     if (m_isTransitioning) {
258         qWarning() << "[SceneManager] 正在过渡中，无法切换场景";
259         return false;
260     }
261
262     Scene* newScene = m_scenes.value(sceneName);
263     QString oldSceneName = m_currentSceneName;
264
265     // 如果是同一个场景，不处理
266     if (m_currentScene == newScene && !isPush) {
267         qDebug() << "[SceneManager] 已经在目标场景:" << sceneName;
268         return true;
269     }
270
271     emit sceneChanging(oldSceneName, sceneName);
272
273     // 加载新场景（如果未加载）
274     if (!newScene->isLoading()) {
275         if (!newScene->load()) {
276             qCritical() << "[SceneManager] 场景加载失败:" << sceneName;
277             return false;
278         }
279     }
280
281     // 处理旧场景
282     if (m_currentScene && !isPush) {
283         // 非push操作，旧场景退出
284         if (useTransition) {
285             m_isTransitioning = true;
286             m_currentScene->startExitTransition(DEFAULT_TRANSITION_DURATION);
287             // TODO: 等待过渡完成
288             // 简化处理：直接切换
289             m_isTransitioning = false;
290         }
```

```
291         m_currentScene->onExit();
292     } else if (m_currentScene && isPush) {
293         // push操作, 旧场景只是暂停
294         // 不调用onExit, 保持状态
295     }
296
297     // 如果不是push, 清空栈
298     if (!isPush) {
299         clearSceneStack();
300     }
301
302     // 设置新场景
303     m_sceneStack.push(newScene);
304     m_currentScene = newScene;
305     m_currentSceneName = sceneName;
306
307     // 设置视图
308     m_view->setScene(newScene->getGraphicsScene());
309
310     // 进入新场景
311     if (useTransition) {
312         m_isTransitioning = true;
313         newScene->startEnterTransition(DEFAULT_TRANSITION_DURATION);
314         // TODO: 等待过渡完成
315         // 简化处理: 直接激活
316         m_isTransitioning = false;
317     }
318
319     if (!newScene->isActive()) {
320         newScene->onEnter();
321     }
322
323     if (isPush) {
324         emit scenePushed(sceneName);
325     }
326     emit sceneChanged(oldSceneName, sceneName);
327
328     qDebug() << "[SceneManager]" << (isPush ? "推送场景" : "切换场景")
329         << ":" << oldSceneName << "->" << sceneName;
330
331     return true;
332 }
333
334 void SceneManager::finishSceneChange()
335 {
336     m_isTransitioning = false;
337     // TODO: 完成场景切换后的处理
```

3.4 更新CMakeLists.txt

将scene.h和scene.cpp添加到core模块的CMakeLists.txt中：

src/core/CMakeLists.txt - 新增scene

```
1  add_library(core STATIC
2      # ... 其他文件 ...
3      scene.h          # 新增
4      scene.cpp        # 新增
5      eventManager.h
6      eventManager.cpp
7      gameEvent.h
8  )
```

3.5 创建测试场景

为了验证场景系统，我们创建一个简单的测试场景：

操作步骤

1. 在 `src/` 下创建 `scenes/` 目录
2. 创建TestScene类（继承自Scene）
3. 在GameEngine中注册并切换到测试场景
4. 编译运行验证

（这部分可以作为选做练习，或者等到第四阶段UI开发时再实现具体场景）

验收标准

- ☐ 编译无错误
- ☐ Scene基类创建完成，生命周期方法完整
- ☐ SceneManager支持Scene基类
- ☐ 场景注册/移除功能正常
- ☐ 场景切换功能正常
- ☐ 场景栈（push/pop）功能正常
- ☐ 场景过渡效果框架完整
- ☐ CMakeLists.txt已更新

五、步骤4：完善资源管理系统



目标：完善资源管理系统，实现资源引用计数、异步加载队列、缓存策略优化

修改文件：resourceManager.h / resourceManager.cpp

4.1 需要完善的功能

第一阶段的ResourceManager有基础的加载和缓存功能，但还需要完善：

- **引用计数：**追踪资源使用情况，自动释放未使用的资源
- **异步加载：**后台线程加载资源，不阻塞游戏主线程
- **加载队列：**批量加载资源，支持优先级
- **资源组：**按场景/功能分组管理资源，便于批量加载/卸载
- **更多资源类型：**字体、动画、瓦片集等

4.2 修改ResourceManager头文件

操作步骤

1. 打开 `src/core/resourceManager.h`
2. 添加资源引用计数结构体
3. 添加资源组管理
4. 添加异步加载相关成员
5. 添加新的公有方法
6. 保存文件

由于ResourceManager的完善代码量较大，这里给出核心的修改点：

resourceManager.h 核心新增内容

resourceManager.h - 新增内容

```
1 // 资源引用计数结构体
2 struct ResourceInfo {
3     QString path;
4     QPixmap pixmap; // 或其他资源类型
5     int refCount;
6     QString group;
```



```

7     bool isPermanent; // 永久资源（不自动释放）
8 };
9
10 // 资源组结构体
11 struct ResourceGroup {
12     QString name;
13     QStringList resources;
14     bool isLoading;
15 };
16
17 // 新增方法：
18 // 引用计数
19 void addRef(const QString& resourcePath);
20 void releaseRef(const QString& resourcePath);
21 int getRefCount(const QString& resourcePath) const;
22
23 // 资源组管理
24 bool createResourceGroup(const QString& groupName);
25 bool addToGroup(const QString& groupName, const QString& resourcePath);
26 bool loadGroup(const QString& groupName);
27 bool unloadGroup(const QString& groupName);
28
29 // 异步加载
30 void loadAsync(const QString& resourcePath);
31 bool isLoading(const QString& resourcePath) const;
32 float getLoadProgress() const;
33
34 // 资源预加载
35 bool preloadResources(const QStringList& paths);
36
37 // 内存管理
38 void setMemoryLimit(int mb);
39 int getMemoryUsage() const;
40 void releaseUnusedResources();

```

4.3 修改ResourceManager实现

在resourceManager.cpp中实现上述新增的方法。

重点实现：

1. 引用计数的增减逻辑
2. 资源组的批量加载/卸载
3. 异步加载的线程处理（使用QtConcurrent或QThread）

提示：Qt推荐使用QtConcurrent进行异步操作，可以很方便地实现后台加载。

验收标准

- ☐ 编译无错误
 - ☐ 资源引用计数功能正常
 - ☐ 资源组管理功能正常
 - ☐ 异步加载框架完整
 - ☐ 内存管理功能正常
 - ☐ 资源加载失败有错误处理
-

六、步骤5-8：其他系统完善



说明： 以下几个系统的完善思路类似，都是在第一阶段基础上增加更多功能。由于篇幅限制，这里列出每个系统需要完善的要点，你可以按照前面的模式逐步实现。

6.1 步骤5：完善输入系统

需要完善的功能

- **输入上下文：** 不同场景有不同的输入映射（游戏中、菜单中、对话中等）
- **输入事件：** 通过事件系统发布输入事件，各系统订阅而不是直接调用
- **配置文件支持：** 按键映射从配置文件加载，支持自定义
- **输入缓冲：** 按键输入有短暂的缓冲时间，提升操作手感
- **手柄支持：** 支持游戏手柄输入（QGamepad）
- **组合键：** 支持组合按键触发（如Ctrl+S）

主要修改点

1. 添加InputContext枚举或类
2. 输入事件通过EventManager发布
3. 从GameConfig加载按键映射
4. 添加输入缓冲逻辑

6.2 步骤6：完善音频系统

需要完善的功能

- **音频池**：复用音效播放器，减少创建/销毁开销
- **音频组**：按类型分组管理（UI、战斗、环境等），可单独调节音量
- **淡入淡出**：音乐切换时的淡入淡出效果
- **3D音效基础**：根据位置调整左右声道音量
- **音频配置联动**：音量变化通过事件系统通知
- **播放列表**：背景音乐播放列表，支持随机/顺序播放

6.3 步骤7：完善配置和存档系统

配置系统完善

- 配置变更通过事件系统发布
- 配置与UI联动（修改配置立即生效）
- 多套配置方案（画质预设：低/中/高/超高）
- 配置版本管理（升级时迁移旧配置）

存档系统完善

- 存档数据结构标准化
- 存档版本管理（兼容旧版本存档）
- 存档缩略图（保存游戏截图）
- 存档加密（可选）
- 云存档基础框架
- 自动存档触发点（进入新场景、战斗结束等）

6.4 步骤8：完善插件系统

需要完善的功能

- **插件接口定义**：定义标准的插件接口（IPlugin）
- **插件元数据**：插件名称、版本、作者、依赖等信息
- **插件生命周期**：加载、初始化、启用、禁用、卸载
- **插件依赖管理**：处理插件间的依赖关系
- **插件配置**：每个插件有独立的配置
- **热重载**：运行时重新加载插件（调试用）

插件接口示例

IPlugin 接口示例

```
1  class IPlugin
2  {
3  public:
4      virtual ~IPlugin() = default;
5
6      // 插件信息
7      virtual QString getName() const = 0;
8      virtual QString getVersion() const = 0;
9      virtual QString getAuthor() const = 0;
10     virtual QString getDescription() const = 0;
11
12     // 生命周期
13     virtual bool initialize() = 0;
14     virtual void shutdown() = 0;
15
16     // 启用/禁用
17     virtual void onEnabled() {}
18     virtual void onDisabled() {}
19
20     // 依赖
21     virtual QStringList getDependencies() const { return {}; }
22 };
```

七、步骤9：核心引擎集成测试



目标：将所有系统整合在一起，创建测试场景验证各系统功能，建立性能基准

9.1 创建测试场景

创建一个TestScene，用于验证各系统功能：

- 显示FPS和游戏时间
- 测试输入系统（按键响应）
- 测试资源加载（加载/释放图片）
- 测试音频系统（播放音效）
- 测试事件系统（发布/订阅事件）
- 测试场景切换（切换到另一个测试场景）

9.2 性能基准测试

建立性能基准，便于后续优化时对比：

- 空场景FPS基准
- 100个精灵的FPS
- 1000个精灵的FPS
- 内存占用基线
- 启动耗时

9.3 内存泄漏检测

使用工具检测内存泄漏：

- Debug模式下的AddressSanitizer（已在CMake中配置）
- 场景加载/卸载循环测试
- 资源加载/释放循环测试

验收标准

- ☐ 测试场景能正常运行
- ☐ 各系统功能正常
- ☐ 场景切换无崩溃
- ☐ 资源加载/释放无内存泄漏
- ☐ 性能基准数据已记录

八、步骤10：调试工具



目标：实现基础的调试工具，便于后续开发调试

10.1 调试控制台

- ~键打开/关闭调试控制台
- 支持输入调试命令（如切换场景、无敌模式、调整时间缩放等）
- 显示日志输出

10.2 性能统计显示

- FPS显示（当前/平均/最低）
- 帧时间显示
- 内存占用显示
- 各系统耗时统计（可选）

10.3 调试热键

- F1：显示/隐藏调试信息
- F2：切换时间缩放（0.5x / 1x / 2x）
- F3：暂停/继续
- F5：快速保存
- F9：快速读档
- F12：截图

九、第二阶段总结

 **第二阶段完成标志：**核心引擎8大系统全部完善，事件驱动架构建立，各系统解耦通信，有可运行的测试场景和基础调试工具

完成清单

- ☐ 游戏主循环和时间管理完善（deltaTime、固定步长、FPS）
- ☐ 事件系统实现（事件总线、订阅/发布）
- ☐ 场景系统完善（场景基类、生命周期、过渡效果）
- ☐ 资源管理完善（引用计数、异步加载、资源组）
- ☐ 输入系统完善（输入上下文、事件、配置支持）
- ☐ 音频系统完善（音频池、淡入淡出、音频组）
- ☐ 配置和存档系统完善
- ☐ 插件系统基础实现
- ☐ 各系统集成测试通过
- ☐ 基础调试工具实现

核心架构特点

- **事件驱动**：各系统通过事件通信，耦合度低
- **场景生命周期**：标准的加载/进入/更新/退出流程
- **资源管理**：引用计数 + 资源组 + 异步加载
- **时间管理**：deltaTime + 固定时间步长 + 时间缩放
- **可扩展性**：插件系统支持功能扩展

下一步：第三阶段

第三阶段将进入渲染系统开发，实现：

- 2.5D等距视角渲染
- 相机系统完善（跟随、边界、震动）
- 精灵动画系统
- 瓦片地图系统
- 粒子效果系统
- 光照和阴影基础



恭喜！ 第二阶段核心引擎开发完成后，你将拥有一个功能完善的2D游戏引擎基础框架，可以开始制作具体的游戏内容了！

（注：文档部分内容可能由 AI 生成）